PHYB57 · TERM TEST 2 · C1–C21 (through Nov 9)

1 · PYTHON CORE (C1–C5)

Operators: + − * / // % **; 7/2=3.5, 7//2=3, −7//2=−4, 7%2=1, 2**3**2=512, −2**2=−4. **Order:** () → ** → unary − → * / // % → + − → comparisons (< <= > >= == !=) → not → and → or.

Types: int, float (1e3, 3.14), complex (2+5j, complex(2,5)), bool, str, None. type(x); int(3.9)=3; True==1, False==0 → (3>2)+(5>4)=2; any non-zero number is truthy. = assigns (right→left), == compares. x=x+1 updates x; after y=x+1 changing x does not change y.

math: import math → math.sqrt, sin, exp, log (=ln), log10, pi, factorial; help: math.factorial?. **Names:** letters/digits/_ , can't start with digit, case-sensitive, don't shadow built-ins. %whos list vars, del x, %reset.

Strings (immutable): s='Hello World'; len(s)=11; s[0]='H', s[−1]='d'; s[6:11]='World' (end excluded); s[:5], s[::2], s[::−1] reverse; + concat, 'ab'*2; .upper() .lower() .replace('a','b') .split(',') .strip() .find('lo') .count('l'); 'lo' in s; str(3), float('2.5'); f-string f"{x:.2f} {n:5d} {v:.3e} {3*4}".

List (mutable, any types): L=[1,2,'a',[3,4]]; L.append(x), L.insert(i,x), del L[i], L.pop(), L.remove(x), L.sort(), L1+L2, len(L), x in L, L[1:3], L[::2], list('abc'), sum(L). b=a is the same object; copy with a.copy() / a[:].

Tuple (immutable → TypeError on assign): t=(1,2,3), (5,); unpack a,b,c=t; t.count(2), t[1:3]. **Set** (unordered, unique): set([1,2,2,3])={1,2,3}, set('Banana')={'B','a','n'}; .union .intersection .difference .symmetric_difference .issubset .issuperset .add; empty = set() ({} is a dict).

Dict (key→value, keys unique & immutable: str/number/tuple, not list): d={'a':1}; d['b']=2; d['a']; d.keys() .values() .items(); 'a' in d tests keys; dict([('a',1),('b',2)]); del d['a']; missing key → KeyError.

NumPy: a=np.array([[1,4,3],[9,2,7]]) → a.shape=(2,3), a.size=6; np.arange(1,5,1)=[1,2,3,4] (end excl.); np.linspace(a,b,n) = n pts incl. both ends, h=(b−a)/(n−1); np.zeros((2,3)), np.ones(n); index a[0,1], row a[1,:], col a[:,0], a[−1], mask x[x>3]. + − * / ** are element-wise (b*d ≠ matrix product); matrix product np.dot(A,B) or A@B; A.T; np.linalg.inv(A). np.sum, mean, max, abs, sqrt, exp, log, sin, cos, pi, diff; np.vstack([x,np.ones(len(x))]).T; y[:,np.newaxis] → column; np.random.seed(1); np.random.random(n).

```
def f(a, b=2):        # b has a default value
    """docstring"""
    return a+b, a*b   # multiple outputs = tuple
s, p = f(1)           # s=3, p=2 ; f(1, b=5)
sq = lambda x: x**2   # anonymous, ONE expression
sorted(L, key=lambda t: t[1])   # sort by 2nd item
def plus1(f, x): return f(x)+1  # function as argument
plus1(np.sin, np.pi/2) → 2.0 ; plus1(lambda x: x+2, 2) → 5
```

Scope: variables made inside a function are local (own memory block): out=1; f() that sets out=6 inside leaves outer out=1. Reading a global works; assigning to it inside a function without global n → UnboundLocalError (referenced before assignment). **Nested** function: defined inside parent, callable only by parent. No return → returns None; print ≠ return. Wrong arg type → TypeError. **Recursion:** base case + self call: def fact(n): return 1 if n<=1 else n*fact(n−1).

```
if temp < want-5:  s='Heat'     # first True branch runs,
elif temp > want+5: s='AC'      # rest are skipped;
else:               s='Off'     # same indentation in block
y = a if cond else b            # ternary
for i in range(1,4): n += i     # i=1,2,3 → n=6 ; range(n)=0..n−1
for c in 'banana': ...   for k,v in d.items(): ...
for i,x in enumerate(L): ...  for a,b in zip(A,B): ...
while n >= 1: n /= 2; i += 1    # n=8 → i=4, n=0.5
break → leave loop ; continue → next iteration
[x**2 for x in range(5)]              → [0,1,4,9,16]
[x for x in L if x%2==0]              # filter
[i+j for i in range(2) for j in range(3)] → [0,1,2,1,2,3]
{k: v**3 for k,v in d.items()}        # dict comprehension
```

range(a,b,step) end excluded. While = unknown # of iterations; condition never False → infinite loop.

2 · COMPLEXITY & PROFILING (C8)

Count basic operations vs input size n; keep highest-order term, drop constants: $4n^2+n+1 \to \mathbf{O(n^2)}$. Growth: O(1) < O(log n) < O(n) < O(n log n) < O(n²) < O(2ⁿ).

Pattern	O	Pattern	O
one loop to n; iterative Fibonacci	n	loop halving n (n//=2)	log n
two nested loops to n	n^2	recursive fib f(n−1)+f(n−2)	2^n

2× computing power: exponential alg. solves n→n+1 only; polynomial O(n^c) solves $n \to 2^{1/c}\cdot n$. **NP-complete**: no known polynomial-time algorithm (Travelling Salesman, Set Cover) — basis of encryption. **Profiling:** %time stmt (one line, once), %%time (whole cell), %timeit (mean ± std over many loops), cProfile/%prun (per function), %lprun −f func func(args) (line_profiler, per line).

例題｜复杂度（数循环次数） for i in range(n): for j in range(i): s+=1。**套路**：①数最内层语句执行几次 ②按 i 求和：0+1+…+(n−1) = n(n−1)/2 ③只留最高阶、去掉系数 ⇒ **O(n²)**。若内层是 while j>1: j//=2（每次减半）⇒ 每轮 log n，总 O(n log n)；递归每次调用 2 次自己且 n−1 ⇒ O(2ⁿ)。

3 · NUMBER REPRESENTATION & ROUND-OFF (C9)

Base N: digits x powers of N: $121_3 = 1\cdot9+2\cdot3+1 = 16$; $147.3 = 1\cdot10^2+4\cdot10+7+3\cdot10^{-1}$. **Dec→bin:** divide by 2, read remainders bottom→top: 37→18r1→9r0→4r1→2r0→1r0→0r1 ⇒ 100101; 11=1011. **Fraction:** ×2, read integer parts top→down: .625→1.25→0.5→**1.0** ⇒ .101. Add with carry (1+1=10): 100101+10001=110110 (37+17=54). 37×17=1001110101 (629).

$2^0 \ldots 2^{10} = 1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024$; $2^{-1} \ldots 2^{-4} = .5, .25, .125, .0625$.

IEEE-754 double (64 bit): 1 sign s | 11 exponent e | 52 fraction f n = $(-1)^s \cdot 2^{e-1023} \cdot (1+f)$, f = $\Sigma\, b_i 2^{-i} \in [0,1)$

Decode 1 10000000010 1000…0: s=1, e=1026 → 2^3, 1+f=1.5 ⇒ −12.0. **Encode** 15 = $1111_2 = 1.111_2 \times 2^3$ → s=0, e=3+1023=1026=10000000010, f=1110…0.

e=0 → subnormal: $(-1)^s 2^{-1022}(0+f)$. e=2047 (all 1s): f=0 → ±inf, f≠0 → NaN.

max ≈ $2^{1023}(2-2^{-52}) \approx 1.8e308$; min normal $2^{-1022} \approx 2.2e{-}308$; min subnormal $2^{-1074} \approx 4.9e{-}324$. **Overflow → inf; underflow → 0.**

Gap between adjacent floats grows with |x|: np.spacing(x) = 2^{E-52} for $2^E \le x < 2^{E+1}$, spacing(1) = $2^{-52} \approx 2.2e{-}16$ (≈16 sig. digits); spacing(1e9) ≈ 1.2e−7. Adding less than half a gap leaves x unchanged: 1e9 + np.spacing(1e9)/3 == 1e9 → True. **Round-off error** = true value − stored approximation (binary can't represent 0.1, 1/3, π exactly). 0.1+0.2+0.3 != 0.6; 4.9−4.845 = 0.05500000000000004; compare floats with abs(a−b)<tol. **Accumulates:** 1 + 1/3 − 1/3 once = 1.0; repeated 100× → 1.0000000000000002, grows with # of repetitions. **Repeated rounding:** 4.845→4.85→4.9 but direct → 4.8.

例題｜13.25 → IEEE−754（编码五步） ①整数部分除 2 取余：13 = 1101 ②小数部分乘 2 取整：.25→0.5→1.0 ⇒ .01 ③合并并规格化：1101.01 = 1.10101×2^3（小数点左移 3 位，E=3）④exponent e = E+1023 = 1026 = 10000000010 ⑤fraction = 小数点后 10101 后面补 0 补满 52 位；s=0。**反向解码**：读 s、e、f → $(-1)^s \cdot 2^{e-1023} \cdot (1+f)$。**检验**：−6.5 ⇒ s=1, 6.5=1.101×2^2, e=1025。

4 · FILES: TXT / CSV / PICKLE / JSON / HDF5 (C11)

f=open(name, mode): 'r' read (default; file must exist), 'w' write (creates / overwrites), 'a' append, 'r+' read+write, 'w+' write+read (discards old), 'a+' append+read; add b for binary. Always f.close() — or use with (auto-close).

```
f = open('test.txt','w')
for i in range(5): f.write(f"This is line {i}\n")
f.close()
with open('test.txt','a') as f: f.write("more\n")
f = open('test.txt','r'); s = f.read(); f.close() # all → 1 str
f.readline() → one line ; f.readlines() → list of lines
for line in f: print(line.strip())     # strip() drops \n
np.savetxt('a.txt', arr, fmt='%.2f', header='Col1 Col2')
arr = np.loadtxt('a.txt')              # skips '#' header
t, y = np.loadtxt('a.txt', unpack=True)   # columns → vars
np.savetxt('d.csv', data, fmt='%.2f', delimiter=',')
d = np.loadtxt('d.csv', delimiter=',') ; d[:5,:]  # 5 rows
df = pd.read_csv('d.csv') ; df.to_csv('o.csv')    # pandas
pickle.dump(obj, open('f.pkl','wb'))
obj = pickle.load(open('f.pkl','rb'))
json.dump(d, open('f.json','w')); d=json.load(open('f.json','r'))
hf = h5py.File('st.hdf5','w'); hf['/acc/1/data'] = arr
hf['/acc/1'].attrs['dt'] = 0.04 ; hf.close()
hf = h5py.File('st.hdf5','r'); list(hf.keys()) → ['acc','gps']
hf['/acc/1/data'][:] ; hf['/acc/1'].attrs['dt']
```

Format	What / when
TXT/CSV	plain text, universal (Excel, R, MATLAB); CSV = one record per line, fields split by commas; numeric tables
Pickle	serialize (object→byte stream) any Python object, binary 'wb'/'rb'; Python-only, version issues (Py2 file: encoding='latin1') → not for sharing/long-term
JSON	text key–value like a dict (str, num, list, nested); language-independent, compact; APIs / data exchange
HDF5	binary, hierarchical: **groups** (folders) + **datasets** (arrays) + **attributes** (metadata); huge data, fast & parallel I/O; h5py, PyTables

5 · VISUALIZATION (C12)

```
import matplotlib.pyplot as plt
plt.figure(figsize=(10,6))
plt.plot(x, y, 'ro', label='data')  # red circles
plt.plot(x, y2, 'g--', label='fit') # 'b-' line 'g--' dash 'k:'
plt.title('T'); plt.xlabel('x'); plt.ylabel('y')
plt.legend(); plt.grid(); plt.xlim(0,5); plt.show()
plt.scatter(x,y) ; plt.loglog(x,y) ; plt.semilogy(x,y)
plt.savefig('f.png') ; plt.style.use('seaborn-poster')
from mpl_toolkits import mplot3d
ax = plt.axes(projection='3d')
ax.plot3D(x, y, t) ; ax.scatter(x, y, z, c='r') # 3-D line, scatter
X, Y = np.meshgrid(x, y); Z = np.sin(X)*np.cos(Y)
ax.plot_surface(X, Y, Z, cmap=plt.cm.cividis)
import cartopy.crs as ccrs; import cartopy.feature as cf
ax = plt.axes(projection=ccrs.PlateCarree())
ax.coastlines(); ax.gridlines(draw_labels=True)
ax.set_extent([lon0, lon1, lat0, lat1])    # zoom
ax.add_feature(cf.LAND)   # OCEAN BORDERS LAKES RIVERS
import matplotlib.animation as manimation
W = manimation.writers['ffmpeg']
writer = W(fps=15, metadata=dict(title='Movie'))
fig = plt.figure(); plt.plot(x, np.sin(x), 'b-') # background
dot, = plt.plot([], [], 'ro')              # placeholder
with writer.saving(fig, 'out.mp4', 100):  # 100 dpi
    for i in range(n):
        dot.set_data([x[i]], [y[i]]); writer.grab_frame()
```

%matplotlib inline static; %matplotlib notebook interactive (pan/zoom). meshgrid: X,Y have shape (len(y), len(x)). Maps: longitude −180…180 (E–W), latitude −90…90 (N–S); Cartopy replaced Basemap; Folium = interactive web maps. Animation = still frames shown rapidly: ① metadata/writer ② draw background ③ loop: update object + grab frame.

6 · LEAST-SQUARES REGRESSION (C16)

Noisy data (x_i, y_i), i=1…m. Model = linear combination of n linearly independent basis functions: $\hat{y}(x) = \Sigma_{j=1\ldots n}\, \alpha_j f_j(x)$. Residual $e_i = y_i - \hat{y}(x_i)$. Minimize total squared error $\mathbf{E = \Sigma\, e_i^2 = \|Y - A\beta\|_2^2}$.

A = m×n matrix, $A_{ij} = f_j(x_i)$; β = $[\alpha_1 \ldots \alpha_n]^T$; Ŷ = Aβ. Residual ⊥ col(A): $A^T(Y - A\beta) = 0$ ⇒

Normal eqs: $\mathbf{A^TA\,\beta = A^TY \Rightarrow \beta = (A^TA)^{-1}A^TY}$ = pinv(A)·Y (A^TA invertible iff columns of A independent).

Calculus: $\partial E/\partial \alpha_k = 0 \Rightarrow \Sigma_j\, \alpha_j \Sigma_i\, f_j(x_i)f_k(x_i) = \Sigma_i\, f_k(x_i)\, y_i$ for k=1…n — the same system.

Line y = mx + b (f_1=x, f_2=1; A = [x | 1]; N = # points): $m = \frac{N\,\Sigma x_i y_i - \Sigma x_i\, \Sigma y_i}{N\,\Sigma x_i^2 - (\Sigma x_i)^2}$, $b = \bar{y} - m\,\bar{x} = \frac{\Sigma y_i - m\,\Sigma x_i}{N}$. **Through origin** (spring F = kx, one basis f=x): $k = \frac{\Sigma\, x_i F_i}{\Sigma\, x_i^2}$.

A^TA = [[Σx², Σx],[Σx, N]], A^TY = [Σxy, Σy]^T. 2×2 inverse: $\frac{1}{ad-bc}$[[d, −b],[−c,a]].

Ex (0,1),(1,3),(2,2): N=3, Σx=3, Σy=6, Σxy=7, Σx²=5 → m=(21−18)/(15−9)=0.5, b=2−0.5·1=1.5; E = (−.5)²+1²+(−.5)² = 1.5.

解題結構｜最小二乘手算（4 步） ①列表算 N、Σx、Σy、Σxy、Σx²（先写表，不会漏项）②写 A^TA=[[Σx², Σx],[Σx, N]]，A^TY=[Σxy, Σy]^T ③解 2×2（消元或公式 m、b）④算残差 $e_i = y_i - (mx_i + b)$，$E = \Sigma e_i^2$（**检验**：Σe_i=0，且 e ⊥ x）。过原点 ⇒ 只有一个基函数 ⇒ k=Σxy/Σx²。

例題｜幂律线性化 (1,3),(2,12),(4,48) 拟合 y=bx^m。①两边取 ln：ln y = m·ln x + ln b ②新点 (ln x, ln y)=(0, ln3),(ln2, ln3+2ln2),(2ln2, ln3+4ln2) ③斜率 = 2ln2/ln2 = **m=2**，截距 ln b=ln3 ⇒ **b=3**。指数型 y=α$e^{\beta x}$ 则用 (x, ln y)：β=斜率，α=$e^{截距}$。

```
A = np.vstack([x, np.ones(len(x))]).T ; y = y[:, np.newaxis]
ATA = np.dot(A.T, A) ; ATY = np.dot(A.T, y)
alpha = np.dot(np.linalg.inv(ATA), ATY)      # direct
alpha = np.linalg.pinv(A).dot(y)             # pseudo-inverse
alpha = np.linalg.lstsq(A, y, rcond=None)[0]  # lstsq
def func(x, a, b): return a*x + b            # any model
popt, pcov = optimize.curve_fit(func, x, y, p0=[1,1])
perr = np.sqrt(np.diag(pcov))                # 1σ errors
c = np.polyfit(x, y, deg) ; np.polyval(c, x)  # highest power 1st
```

Non-linear models — log trick (linearize, fit a line, recover):

Exponential y = α$e^{\beta x}$ → ln y = βx + ln α: fit (x, ln y) → β = slope, α = exp(intercept).

Power law y = bx^m → ln y = m ln x + ln b: fit (ln x, ln y) → m = slope, b = exp(intercept).

Polynomial $y = a_n x^n + \ldots + a_0$ is linear in the a's → A columns [x^n … x 1] or polyfit. Low order → underfit; high order → RSS↓, R²↑ but overfitting (fits noise, wild between points). **curve_fit**: direct iterative non-linear fit, no transform needed (e.g. sums of exponentials, unknown frequency); give initial guess p0.

7 · INTERPOLATION (C17)

Find f with $f(x_i) = y_i$ — passes through every point; no model assumed; for reliable data, inside the data range only.

Linear ($x_i < x < x_{i+1}$): $\hat{y}(x) = y_i + \frac{(y_{i+1} - y_i)(x - x_i)}{x_{i+1} - x_i}$. Ex x=[0,1,2], y=[1,3,2]: ŷ(1.5) = 3 + (2−3)(.5)/1 = 2.5. f=interp1d(x,y); f(1.5) (scipy.interpolate).

Cubic spline: piecewise cubics $S_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i$ on $[x_i, x_{i+1}]$. n points → n−1 cubics → **4(n−1) unknowns**. Equations: $S_i(x_i) = y_i$ and $S_i(x_{i+1}) = y_{i+1}$ → 2(n−1); $S'_i(x_{i+1}) = S'_{i+1}(x_{i+1})$ → n−2; $S''_i(x_{i+1}) = S''_{i+1}(x_{i+1})$ → n−2; + 2 boundary conditions — **natural**: $S''_1(x_1) = 0$, $S''_{n-1}(x_n) = 0$. Total 4(n−1) ✓. Smooth (C²).

f=CubicSpline(x,y,bc_type='natural'); f(1.5).

Lagrange (one polynomial, degree n−1): $L(x) = \Sigma_i\, y_i P_i(x)$, $P_i(x) = \Pi_{j \ne i} \frac{x - x_j}{x_i - x_j}$,

$P_i(x) = 1$ if i=j else 0.

Ex x=[0,1,2], y=[1,3,2]: $P_1 = ½(x-1)(x-2)$, $P_2 = -x(x-2)$, $P_3 = ½x(x-1)$ → L = $1\cdot P_1 + 3\cdot P_2 + 2\cdot P_3 = -1.5x^2 + 3.5x + 1$; L(1.5)=2.875. f=lagrange(x,y).

Newton: $f(x) = a_0 + a_1(x-x_0) + a_2(x-x_0)(x-x_1) + \ldots + a_n(x-x_0)\cdots(x-x_{n-1})$; $a_k = f[x_0, \ldots, x_k]$ = **top row** of divided-difference table:

$f[x_i, x_{i+1}] = \frac{y_{i+1} - y_i}{x_{i+1} - x_i}$, $f[x_i, \ldots, x_{i+k}] = \frac{f[x_{i+1}, \ldots, x_{i+k}] - f[x_i, \ldots, x_{i+k-1}]}{x_{i+k} - x_i}$. Same polynomial as Lagrange; adding a point only adds one new term (old a's kept).

x	y	1st	2nd	3rd
−5	2	**(6+2)/4 = 2**	**(−5−2)/5 = −1.4**	**(2+1.4)/7 = 0.4857**
−1	6	(1−6)/1 = −5	(1+5)/3 = 2	
0	1	(3−1)/2 = 1		
2	3			

⇒ f(x) = −2 + 2(x+5) − 1.4(x+5)(x+1) + 0.4857(x+5)(x+1)x.

```
coef = np.zeros([n,n]); coef[:,0] = y
for j in range(1,n):
    for i in range(n-j):
        coef[i][j] = (coef[i+1][j-1]-coef[i][j-1]) / (x[i+j]-x[i])
a = coef[0,:]                  # evaluate (nested / Horner):
p = a[n]                       # here n = len(x)−1
for k in range(1,n+1): p = a[n-k] + (x - x[n-k])*p
```

Tut 7 (2,5),(4,9),(6,15) → f(3): linear 7; Newton a=[5, 2, 0.25] → 5+2(1)+.25(1)(−1) = 6.75; natural spline 6.8125.

解題結構｜插值題怎么答 ①先看点的个数 n，多项式次数 = n−1 ②**Lagrange**：每个 P_i 分子=其他 (x−x_i)，分母=代入 x_i，求值时检验 ΣP_i=1 ③**Newton**：画均差表，系数取**第一行**；加新点只在表底多算一斜行，旧系数不变 ④**样条**：未知数 4(n−1)；方程 = 过点 2(n−1) + 一阶导连续 (n−2) + 二阶导连续 (n−2) + 边界 2 ⑤手算最后用另一种方法验算同一点。

8 · ROOT FINDING (C19)

Root x_r: $f(x_r) = 0$. **Tolerance** = acceptable error; converged when error < tol. Error metrics: **|f(x)|** (how close to 0) or **$|x_{i+1} - x_i|$** (change between guesses). Pitfall: f = 1/x has no root, yet |f(x)| < tol for huge x → false convergence (fsolve stops at max # of function evaluations, maxfev). Always check both the error size and algorithm behaviour.

Bisection (Intermediate Value Thm: f continuous on [a,b], f(a)·f(b) < 0 ⇒ root inside):

① check opposite signs (else raise error) ② m = (a+b)/2 ③ if |f(m)| < tol → return m; if sign f(a) = sign f(m) → a = m; else b = m ④ repeat.

Always converges, but **linearly**: bracket halves each step, error after n steps ≤ (b−a)/2^n → n ≥ $\log_2$((b−a)/tol).

Ex f=x²−2 on [0,2]: m=1 (−1) → [1,2]; 1.5 (+.25) → [1,1.5]; 1.25 (−.44) → [1.25,1.5]; 1.375 (−.11) → [1.375,1.5]; 1.4375 (+.066 < 0.1 ✓). tol=0.1 → 1.4375 (5 it.); tol=.01 → 1.4140625 (8 it.); 1e−6 → 21 it.

```
def my_bisection(f, a, b, tol):
    if np.sign(f(a)) == np.sign(f(b)):
        raise Exception("a and b do not bound a root")
    m = (a + b)/2
    if np.abs(f(m)) < tol: return m
    elif np.sign(f(a)) == np.sign(f(m)):
        return my_bisection(f, m, b, tol)     # root in [m,b]
    else: return my_bisection(f, a, m, tol)   # root in [a,m]
```

Newton–Raphson (tangent line: 0 = f(x₀) + f′(x₀)(x₁−x₀)): $\mathbf{x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}}$

Quadratic convergence near root (correct digits ≈ double each step) — faster than bisection when x_0 is close. Limits: sensitive to x_0; f′ ≈ 0 → huge/unstable step (÷0); may jump to another root, oscillate or diverge; needs f′.

Ex x²−2, x_0=1.5: $x_1 = 1.5 - .25/3 = 1.41667$; $x_2 = 1.414216$ (4 it. for 1e−6). √a: x ← ½(x + a/x).

```
def my_newton(f, df, x0, tol):
    if abs(f(x0)) < tol: return x0
    return my_newton(f, df, x0 - f(x0)/df(x0), tol)
from scipy.optimize import fsolve
f = lambda x: x**3 - 100*x**2 - x + 100    # roots −1, 1, 100
fsolve(f, [2, 80]) → [1., 100.]            # one root per guess
r,info,ier,msg = fsolve(f, x0, full_output=True) # ier=1 ok
```

解題結構｜求根题怎么答 **二分**：①验证 f(a)f(b)<0 ②列表 a,b,m,f(m) 每行一次 ③停止条件 |f(m)|<tol（注意题目用哪个判据）④迭代次数 n ≥ $\log_2$((b−a)/tol）。**Newton**：①先写 f′ ②代入 $x_{k+1} = x_k - f/f'$ ③用分数保持精确 ④f′≈0 或来回跳（如 x^3-2x+2，x_0=0 在 0↔1 循环）⇒ 不收敛。**选方法**：有区间要稳 ⇒ 二分；有导数且初值好 ⇒ Newton（快）。

9 · NUMERICAL DIFFERENTIATION (C20)

Grid $x_j = x_0 + jh$, spacing h. **Taylor:** $f(x_j \pm h) = f \pm hf' + \frac{h^2}{2}f'' \pm \frac{h^3}{6}f''' + \frac{h^4}{24}f'''' \pm \ldots$ (all at x_j)

Scheme	Formula for f′(x_j)	Leading error	Order
Forward	$[f(x_{j+1}) - f(x_j)] / h$	−(h/2) f″	O(h)
Backward	$[f(x_j) - f(x_{j-1})] / h$	+(h/2) f″	O(h)
Central	$[f(x_{j+1}) - f(x_{j-1})] / 2h$	−(h²/6) f‴	**O(h²)**
f″ central	$[f(x_{j+1}) - 2f(x_j) + f(x_{j-1})] / h^2$	−(h²/12) f⁗	**O(h²)**

Derive: forward = solve Eq(+h) for f′. Central = Eq(+h) − Eq(−h): even terms cancel → 2hf′ + (h³/3)f‴. f″ = Eq(+h) + Eq(−h): odd terms cancel → 2f + h²f″ + (h⁴/12)f⁗. O(h²): halving h cuts error by 2^p (fwd ÷2, central ÷4); on a log-log plot of error vs h the **slope = p** (log err = p log h + log C). Central & f″ formulas are exact for quadratics (f=x²+3x: f′(2)=7, f″(2)=2).

```
h = 0.1; x = np.arange(0, 2*np.pi, h); y = np.cos(x)
fwd = np.diff(y)/h      # = (y[1:]−y[:−1])/h → on x[:−1]
cen = (y[2:] − y[:−2])/(2*h)              # → x[1:−1]
d2  = (y[2:] − 2*y[1:−1] + y[:−2])/h**2  # → x[1:−1]
max_err = max(abs(−np.sin(x[:−1]) − fwd))  # ≈ 0.05 ≈ h/2
```

np.diff output has length n−1. cos, h=0.1: max error fwd 0.04998, central 0.00167 (≈ h²/6).

Noise: f = cos x + ε sin(ωx), ε=0.01, ω=100 → f′ = −sin x + **εω** cos(ωx): 1 % noise becomes amplitude εω = 1 = signal size. Differentiation amplifies high-frequency noise (finite diff: $\Delta y_{noise}/h$); worse for higher derivatives → smooth/fit before differentiating.

例題｜f=x^3 在 x=1，h=0.1（f(0.9)=.729, f(1)=1, f(1.1)=1.331） ①Forward (1.331−1)/.1=**3.31**（误差 .31≈h/2f″=.30）②Backward (1−.729)/.1=**2.71** ③Central (1.331−.729)/.2=**3.01**（误差 .01=(h²/6)f‴）④f″: (1.331−2+.729)/.01=**6.00**。**套路**：真值 f′=3 ⇒ 误差 ⇒ 对照首项系数 ⇒ h 减半：Forward 误差 ÷2，Central ÷4。

10 · NUMERICAL INTEGRATION (C21)

n grid points $x_0 \ldots x_{n-1}$ on [a,b] → n−1 sub-intervals, **h = (b−a)/(n−1)**; $y_i = (x_i + x_{i+1})/2$.

Rule	$\int_a^b f\,dx \approx$	1 interval	Total
Left Riemann	$h\,\Sigma_{i=0\ldots n-2}\, f(x_i)$	O(h²)	O(h)
Right Riemann	$h\,\Sigma_{i=1\ldots n-1}\, f(x_i)$	O(h²)	O(h)
Midpoint	$h\,\Sigma_{i=0\ldots n-2}\, f(y_i)$	O(h³)	**O(h²)**
Trapezoid	$\frac{h}{2}[f(x_0) + 2\Sigma_{i=1\ldots n-2} f(x_i) + f(x_{n-1})]$	O(h³)	**O(h²)**
Simpson ⅓	$\frac{h}{3}[f(x_0) + 4\Sigma_{\text{odd } i} f(x_i) + 2\Sigma_{\text{even } i=0,n-1} f(x_i) + f(x_{n-1})]$	O(h⁵)	**O(h⁴)**

Why total = one order lower: # intervals = (b−a)/h, so N·O(h^{p+1}) = O(h^p). **Midpoint:** expand about y_i — the odd (x−y_i) term integrates to 0 by symmetry ⇒ hf(y_i) + (h^3/24)f″, needs f at midpoints (not possible with fixed tabulated data). **Trapezoid** = area (h/2)[f(x_i)+f(x_{i+1})] = average of left & right sums; uses grid data only; exact for straight lines; interior points counted twice. Error ≈ −(b−a)h²f″/12 (midpoint: + (b−a)h²f″/24, half the size, opposite sign). *(Slide C21 says trapezoid O(h) total; but T: O(h²).)*

Simpson: fit a parabola (Lagrange polynomial) through 3 points (x_{i-1}, x_i, x_{i+1}) over two sub-intervals and integrate exactly: (h/3)[f_{i-1} + 4f_i + f_{i+1}]. Needs an **odd # of points** (even # of intervals). Weights 1,4,2,4,…,2,4,1. Exact for polynomials up to degree 3 (cubic term cancels by symmetry). Error ≈ −(b−a)h⁴f⁗/180.

$\int_0^\pi \sin x\,dx = 2$, n = 11 (h = π/10): Left = Right = Trapezoid = 1.9835 (err .0165); Midpoint = 2.0082 (err −.0082); Simpson = 2.00011 (err −1.1e−4, ~150× better than trapezoid). **Tut 7:** $\int_0^2 (x^2+1)dx = 14/3 = 4.6667$, n=21, h=.1: right 4.87, mid 4.665, trap 4.67, Simpson 4.6667 (exact).

```
a, b, n = 0, np.pi, 11
h = (b−a)/(n−1); x = np.linspace(a, b, n); f = np.sin(x)
I_left  = h*sum(f[:n−1])  ;  I_right = h*sum(f[1:])
I_mid   = h*sum(np.sin((x[:n−1] + x[1:])/2))
I_trap  = (h/2)*(f[0] + 2*sum(f[1:n−1]) + f[n−1])
I_simp  = (h/3)*(f[0] + 4*sum(f[1:n−1:2])
                 + 2*sum(f[2:n−2:2]) + f[n−1])
from scipy.integrate import trapezoid, cumulative_trapezoid, quad
trapezoid(f, x)                       # old name: trapz(f, x)
F = cumulative_trapezoid(f, x, initial=0)   # running ∫ (cumtrapz)
I, err = quad(np.sin, 0, np.pi)       # (func, a, b) → value, err
```

trapezoid/cumulative_trapezoid take arrays (y, x); cumulative gives F(X)=$\int_a^X$ f dt at every grid point (sin → 1−cos x; without initial=0 output has n−1 points). quad takes a function + limits, adaptive quadrature, returns (integral ≈ 2.0, abs-error bound ≈ 2.2e−14).

例題｜f=x^3 在 [0,2]，n=5，h=0.5（f: 0, .125, 1, 3.375, 8；真值 4） 套路：①h=(b−a)/(n−1) ②列出 f 值 ③代公式。**Left**=.5(0+.125+1+3.375)=**2.25**；**Right**=.5(.125+1+3.375+8)=**6.25**；**Trap**=.25[0+2(4.5)+8]=**4.25**（=(L+R)/2）；**Simpson**=(.5/3)[0+4(.125+3.375)+2(1)+8]=4（三次以内精确）；**Midpoint**（只能用 h=1）=1·(.125+3.375)=**3.5**。h 减半：Trap 误差 ÷4（.25→.0625，得 4.0625）。

11 · NO-CALCULATOR AIDS

√2=1.4142, √3=1.7321, √5=2.2361, √10=3.1623; π=3.1416, π/10=.31416, π^2=9.870; e=2.7183, e^2=7.389, e^{-1}=.3679; 2^{10}=1024; 2^{-2}=.25; $\log_{10}2$=0.3010; $\log_2 10$=3.3219; $2^{10} \approx 10^3$, $2^{-52} \approx 2.2e{-}16$.

θ	0	π/6	π/4	π/3	π/2	π
sin	0	.5	.7071	.8660	1	0
cos	1	.8660	.7071	.5	0	−1

Taylor about a: $f(x) = \Sigma\, f^{(n)}(a)(x-a)^n/n!$. $e^x = 1 + x + x^2/2 + x^3/6 \ldots$; sin x = x − x^3/6 + x^5/120…; cos x = 1 − x^2/2 + x^4/24… ; ln(1+x) = x − x^2/2 + x^3/3…; $(1+x)^p \approx 1 + px$. $\log_a x = \ln x / \ln 2$.